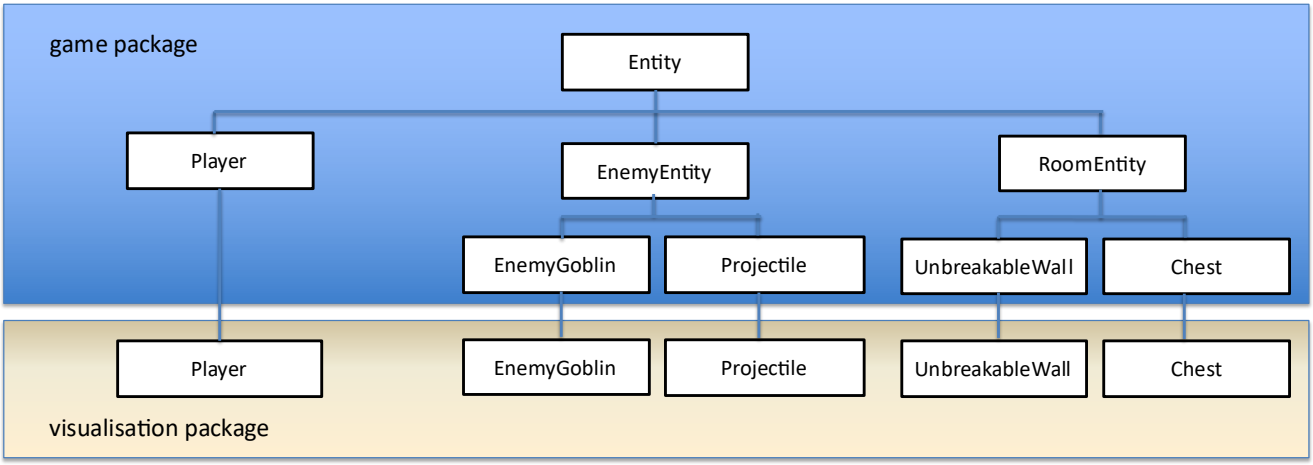
Java Project 2025-2026

Goal of this project:

Create a 2D Top-Down Tower Defence game in Java and follow these guidelines:

Game logic:

- Provide a clear **separation between game logic and game presentation**. Do this by encapsulating all game objects and logic, except their presentation and user-interaction, in a separate **game package** (e.g.: be.uantwerpen.fti.ei.geavanceerde.towerdefence).
  The goal of this separation is to provide a very simple way of writing a completely new visual presentation based on the same logic and structure. So you might have two completely different looking games, based on the same game logic package.
- Design a hierarchy of game entities (obstacles, bonuses, projectiles, etc.) and their interactions (collision control, position control, etc.). Be as creative & fancy as you like. A basic entity class hierarchy might be, for instance, implemented as in this figure:



Specifications:

- Use one single object (might be a "singleton" design pattern) to represent **the game**: a self-contained object with a list of game's entities, a main loop to check for collisions, interactions and states, a facility to keep scores, creation and deletion of game's entities depending on the needs, etc.
- A "game object" **must not** contain any calls to visualization libraries, interactive keyboard control, etc.
- Use the **"abstract factory"** pattern to create entities inside the game object to achieve the separation between game logic and presentation.
- Create a simple way to time events: a basic "Stopwatch" class for keeping time (in milliseconds) between two "ticks" might be a good idea. (Because not every computer runs at the same speed and yet, the visualized entities should move with the same speed on all computers.)
- The game field is a 2D world which is independent from the visualization (i.e. different coordinate systems!). Use a coordinate system with 'Double'-type coordinates for your game. The "visible" game world is a window that is shown on the screen.
- A score indication should be visible during the game.
- Your game should contain **at least one system** designed in a **Data-Oriented** fashion, using an **Entity-Component-System architecture**.
- In the game logic, the usage of 'null' values needs to be **avoided**.
- **At least one** useful implementation of the **Java Streams API** should be implemented.

- Your game logic should include **at least one game script** that is written in **Lua** (e.g. enemy AI, physics, player movements, game events, etc.).
- A configuration file with game parameters should be included (e.g. window size, selected level, lives, etc.).
- The game should be playable on other systems **without any code modifications**.
- In a tower defence game, the player is tasked with defending a base or strategic objective against waves of incoming enemies that follow predefined paths across the map. The game ends when the player successfully survives all enemy waves, or when the base loses all its life points due to enemies reaching it. Enemies enter the map from one or more spawn points and move along fixed or semi-fixed routes toward the player's base. The player cannot directly attack enemies but must strategically place defensive towers on designated build locations across the map. These towers automatically detect and attack enemies within their range. Different types of towers can have different behaviors such as rapid low-damage attacks, slow high-damage attacks, splash damage, or special effects like slowing enemies. Each enemy that reaches the base reduces its life points. If the base reaches zero life points, the game is lost. Destroying enemies grants the player score and/or resources that can be used to build new towers or upgrade existing ones. Players must therefore balance resource management and strategic placement to efficiently stop increasingly difficult enemy waves. Enemy waves should gradually increase in difficulty, for example through higher health, faster movement, or special enemy abilities (e.g. armored enemies, flying enemies, or enemies that temporarily boost nearby units). The behavior of enemies and towers should include some non-trivial mechanics (for example different targeting strategies, resistances, or movement patterns), but the focus should remain on gameplay mechanics rather than complex artificial intelligence. The map may include various environmental elements such as obstacles that block tower placement, terrain features that influence enemy movement, or special tiles that enhance tower performance. Additional bonuses could appear during the game, such as temporary damage boosts, extra resources, or emergency abilities that help the player survive difficult waves. The player's objective is to survive all waves while maintaining the base's health and achieving the highest possible score by efficiently eliminating enemies and managing defenses.
  A simple description can be found on Wikipedia: https://en.wikipedia.org/wiki/Tower_defense
  Create your own version of the game with at least two levels (configurable or predefined).
  Inspiration can be found in existing examples of games on the internet.

Visual & interactive implementation:

- Use the Java2d library (java.awt.Graphics2D) in your "visual" Implementation of the game's objects. Use the Java event library for "interactive" keyboard input (java.awt.KeyEvent, etc.).
  Some information can be found at:
  https://docs.oracle.com/javase/tutorial/2d/index.html
  https://www.iitk.ac.in/esc101/05Aug/tutorial/2d/index.html
  https://zetcode.com/gfx/java2d/
  Use separate packages to encapsulate the "visual" and "interactive" implementations of your game objects (e.g.: J2d).
- In your visual implementation, you can use your own images or you can find some online. Restrict yourself to a 2D representation. You can use transparency or animations.
- Everything put together might look like:

Practical:

- A basic **working** implementation of the package with basic game logic (and more importantly: following clear design considerations and required design patterns, i.e. abstract factory and singleton) & visual representation in Java2D are required to get a passing grade, e.g. proper code commenting & documentation (javadoc) of your API, logical structure of the code & code base (directories), proper class construction that proves you "understand" Java are obviously implied! Including a configuration file (see xml or Properties library java.util.Properties) and Lua game script.
- Extensions are **up to you**:
  - More enemies or other enemies
  - Different game-modes
  - Extra levels (stored in files or generated)
  - More advanced interactions
  - Advanced power-ups
  - Moving obstacles
  - Debris, special effects, particles, etc.
  - Collectables
  - ...
- A portfolio that logs the usage of generated code from LLMs in your project is mandatory.
- Good luck! If you have any questions, remarks or comments feel free to ask during course hours.